# S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

## Practical 07

**Aim:** Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

**Name: Suhani Birkhede**

**USN: CD24064**

**Semester / Year:**

**Academic Session:**

**Date of Performance:**

**Date of Submission:**

## ❖ CODE :

```
GNU nano 8.7                                safe.c                                Modified
#include <stdio.h>
#include <stdbool.h>

#define P 5   // Number of processes
#define R 4   // Number of resources

int main() {

    int allocation[P][R];
    int maximum[P][R];
    int need[P][R];
    int available[R];

    int i, j;

    printf("Enter Allocation Matrix (5 processes x 4 resources):\n");
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("\nEnter Maximum Matrix (5 processes x 4 resources):\n");
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            scanf("%d", &maximum[i][j]);
        }
    }

    printf("\nEnter Available Resources (4 values):\n");
    for(j = 0; j < R; j++) {
        scanf("%d", &available[j]);
    }

    // Calculate Need matrix
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            need[i][j] = maximum[i][j] - allocation[i][j];
        }
    }

    printf("\nNeed Matrix:\n");
    for(i = 0; i < P; i++) {
        for(j = 0; j < R; j++) {
            printf("%d ", need[i][j]);
        }

^G Help      ^O Write Out   ^F Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark   M-] To Bracket  M-B Previous   ^ Back
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line  M-E Redo     M-6 Copy       ^B Where Was    M-F Next       ^ Forward
```

```
GNU nano 8.7                                safe.c                                Modified
            for(j = 0; j < R; j++) {
                if(need[i][j] > work[j]) {
                    possible = false;
                    break;
                }
            }

            if(possible) {
                for(j = 0; j < R; j++) {
                    work[j] += allocation[i][j];
                }

                safeSequence[count++] = i;
                finish[i] = true;
                found = true;
            }
        }
    }

    if(!found) {
        printf("\nSystem is NOT in a safe state.\n");
        return 0;
    }
}

printf("\nSystem is in a SAFE state.\nSafe Sequence:\n");

char *processNames[P] = {
    "Google Drive",
    "Firefox",
    "Word Processor",
    "Excel",
    "PowerPoint"
};

for(i = 0; i < P; i++) {
    printf("%s", processNames[safeSequence[i]]);
    if(i != P - 1)
        printf(" -> ");
}

printf("\n");

return 0;
}

^G Help      ^O Write Out   ^F Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark   M-] To Bracket  M-B Previous   ^ Back
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line  M-E Redo     M-6 Copy       ^B Where Was    M-F Next       ^ Forward
```

## ❖ OUTPUT :

```
Student@DESKTOP-JJ8CFNB MINGW64 ~
$ nano safe.c

Student@DESKTOP-JJ8CFNB MINGW64 ~
$ gcc safe.c -o safe

Student@DESKTOP-JJ8CFNB MINGW64 ~
$ ./safe
Enter Allocation Matrix (5 processes x 4 resources):
2
3
4
1
5
2
3
4
5
2
1
2
3
4
1
2
3
4
5
5

Enter Maximum Matrix (5 processes x 4 resources):
2
4
2
4
3
2
1
3
2
1
4
2
1
3
1
34
```

```
3
4
1
2
3
4
5
5

Enter Maximum Matrix (5 processes x 4 resources):
2
4
2
4
3
2
1
3
2
1
4
2
1
3
1
34


1
1
1
1

Enter Available Resources (4 values):
2
3
4
5

Need Matrix:
0 1 -2 3
-2 0 -2 -1
-3 -1 3 0
-2 -1 0 32
-2 -3 -4 -4

System is NOT in a safe state.

Student@DESKTOP-JJ8CFNB MINGW64 ~
$ |
```